Música — App JavaFX

Academia de Música SonoRitmo

> **Condiciones de la práctica (igual que la evaluación)**
> Trabajo individual, sin internet ni IA · 3 bloques de tiempo · el programa debe compilar y ejecutarse · se entrega el proyecto Maven completo en un ZIP con tu nombre completo. Una solución que no compila no evidencia ningún indicador.

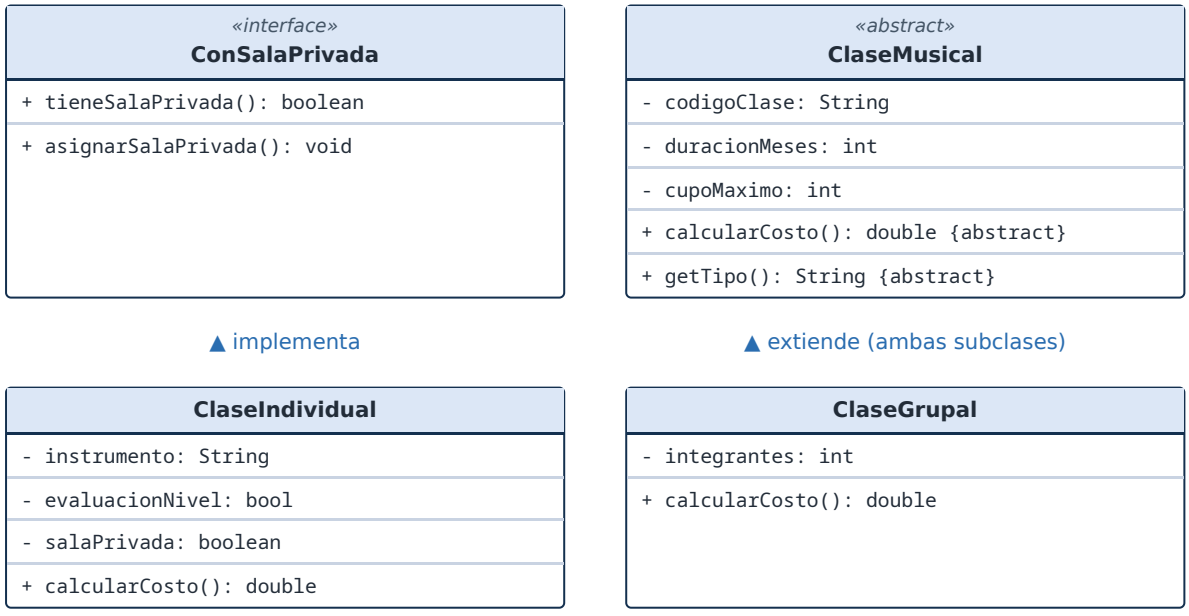
1. Contexto del caso

La Academia de Música SonoRitmo ya cuenta con el modelo de clases desarrollado en la evaluación anterior: el sistema distingue clases individuales y clases grupales, calcula el costo de cada clase según su tipo y permite asignar una sala de ensayo privada a las clases individuales. Sin embargo, ese sistema solo funciona por consola y la información se pierde cada vez que el programa se cierra.

La dirección ha solicitado transformarlo en una aplicación de escritorio de una sola ventana, que permita al personal administrativo registrar y eliminar clases, y que conserve la información en archivos para que esté disponible la próxima vez que se abra la aplicación.

Proyecto base

Partes desde **SonoRitmo_Base.zip**: un proyecto Maven con la configuración inicial, la clase de arranque y el modelo ya implementado y probado, correspondiente al diagrama siguiente. Tu trabajo consiste en construir sobre ese modelo la interfaz gráfica y el almacenamiento en archivos. **No debes reescribir ni reemplazar las clases entregadas.**



El modelo entregado ya aplica las siguientes reglas:

- Clase Individual: costo base de $60.000. Si el estudiante no rindió una evaluación de nivel, el costo se incrementa en un 15%.
- Clase Grupal: costo base de $25.000. Si la cantidad de integrantes supera 6, el costo se incrementa en un 10%.

Reglas de los datos comunes en el modelo: El código de clase no puede ser nulo ni vacío. La duración debe estar entre 1 y 12 meses. El cupo máximo debe ser mayor que cero. Si un dato no cumple, el modelo lo rechaza con un mensaje que explica el motivo.

2. Estructura del proyecto

La solución debe separar responsabilidades según el patrón **Modelo-Vista-Controlador**, con una capa de acceso a datos basada en el patrón **DAO/Repository**. El proyecto debe contener, como mínimo:

- **Configuración del proyecto:** el proyecto base ya incluye las dependencias de JavaFX y la clase de arranque. Debes agregar al `pom.xml` la dependencia de la librería **Gson**, necesaria para trabajar con archivos JSON.
- **Distribución de paquetes y responsabilidades** acorde al patrón: modelo (entregado), vista (FXML), controlador, DAO y Repository en paquetes diferenciados.

> **Reglas que la pauta exige**
> La interfaz se declara en **FXML** (no se acepta una interfaz construida íntegramente por código); DAO y Repository trabajan con colecciones de dominio **sin depender de JavaFX**; la tabla se alimenta de una **colección observable** que adapta la presentación.

3. Ventana de la aplicación

Al ejecutar la aplicación se abre una única ventana, desde la cual se realizan todas las acciones. La ventana debe contener:

- Un **formulario de registro** con los campos descritos más abajo y un botón **Registrar**.
- Una **tabla** con los registros y las columnas Tipo, Código, Duración (meses), Cupo, Costo. El costo que se muestra es el que calcula el modelo.
- Un botón **Eliminar**, que quita del registro la clase seleccionada en la tabla. Si no hay una fila seleccionada, la aplicación informa que se debe seleccionar una clase y no realiza ningún cambio.
- Un **área de mensajes** dentro de la misma ventana, donde se informa el resultado de cada acción: registro exitoso, dato no válido, eliminación, falta de selección o error al leer o guardar los archivos. Cada mensaje reemplaza al anterior. **Los mensajes no se muestran por consola.**

Campos del formulario

- Un selector de tipo con las opciones Clase Individual y Clase Grupal.
- Datos comunes: código de clase, duración (meses), cupo máximo.
- Datos de la clase individual: instrumento principal, rindió evaluación de nivel (sí/no), sala de ensayo privada asignada (sí/no).
- Datos de la clase grupal: integrantes.

Al cambiar el tipo en el selector, solo quedan habilitados los datos específicos del tipo elegido; los datos del otro tipo quedan deshabilitados y no se consideran al registrar.

Al presionar Registrar, la aplicación valida los datos según la sección 4. Si son correctos, registra la clase, la tabla la muestra de inmediato, el área de mensajes informa la confirmación (por ejemplo: “CLA-P01 (Clase Individual) registrada correctamente.”) y el formulario queda vacío para un nuevo registro.

4. Validaciones del formulario

Antes de registrar, el formulario debe verificar los datos ingresados según las siguientes reglas:

- **Tipo:** debe estar seleccionado.
- **Código de clase:** no puede estar vacío ni contener solo espacios.
- **Duración (meses):** número entero entre 1 y 12.
- **Cupo máximo:** número entero mayor que cero.
- **Instrumento principal** (solo clase individual): no puede estar vacío ni contener solo espacios.
- **Integrantes** (solo clase grupal): número entero mayor que cero.

Las reglas de los datos comunes son las mismas que ya aplica el modelo entregado. Si algún dato no es válido:

- la clase no se registra;
- el área de mensajes indica **qué campo** es incorrecto y **por qué**;
- los valores ya ingresados se mantienen para que puedan corregirse;
- la aplicación no se detiene ni muestra errores por consola.

5. Almacenamiento de la información

La información debe conservarse entre una ejecución y otra, en formato **JSON**, en la carpeta raíz del proyecto:

- `clases_individuales.json`: contiene clases individuales registradas.
- `clases_grupales.json`: contiene clases grupales registradas.

El comportamiento esperado es el siguiente:

- Al iniciar, la aplicación carga automáticamente la información de ambos archivos y la muestra en la tabla.
- Si los archivos no existen (por ejemplo, la primera vez que se ejecuta), la aplicación inicia con la tabla vacía y sin mostrar errores.
- Cada vez que se registra o se elimina una clase, los archivos se actualizan automáticamente, sin que el usuario deba presionar un botón para guardar.
- Lo que muestra la tabla y lo guardado en los archivos debe coincidir en todo momento.

6. Datos de prueba y resultado esperado

Registra desde la aplicación los siguientes registros:

Tipo	Código de clase	Duración (meses)	Cupo máximo	Datos específicos	Costo esperado
Clase Individual	CLA-P01	6	1	Instrumento: Piano | Evaluación de nivel: no | Sala privada: sí	$69.000
Clase Individual	CLA-G01	4	1	Instrumento: Guitarra | Evaluación de nivel: sí | Sala privada: no	$60.000
Clase Grupal	CLA-C01	3	12	Integrantes: 8	$27.500
Clase Grupal	CLA-C02	2	10	Integrantes: 5	$25.000

Luego de registrarlos, la ventana debe verse de forma similar a la siguiente:

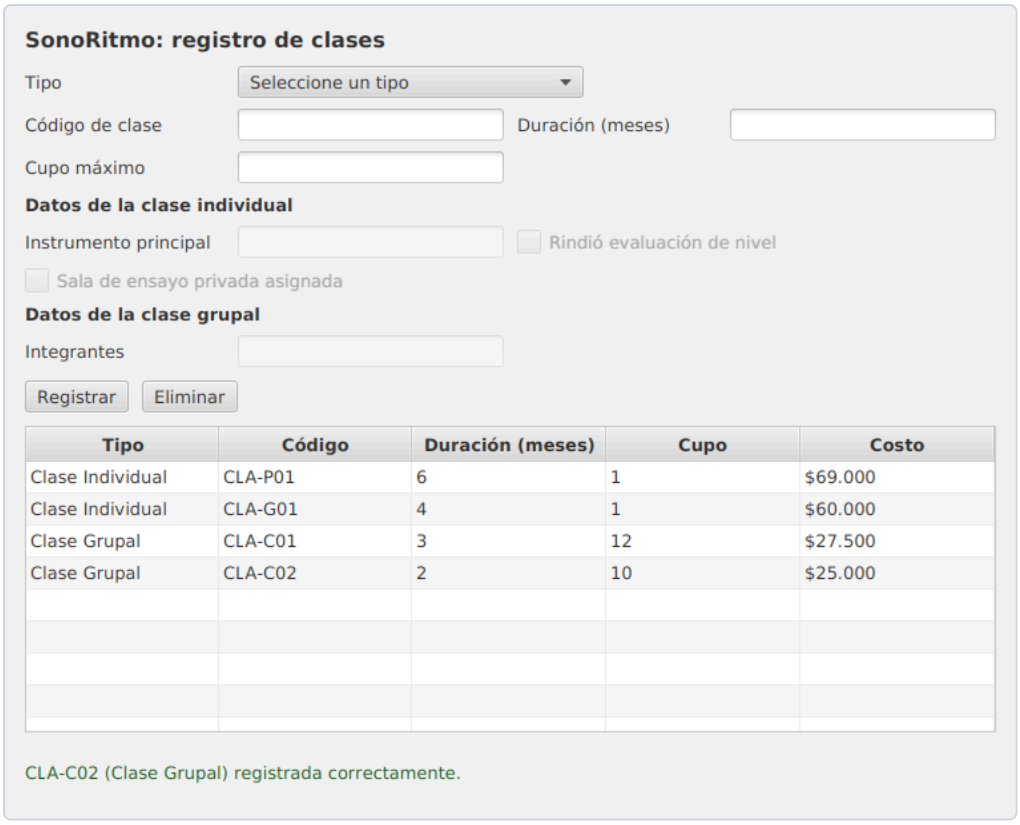


Figura 1. Ventana con los cuatro registros.

Si se presiona Eliminar sin seleccionar una fila, el aviso aparece en la misma ventana:

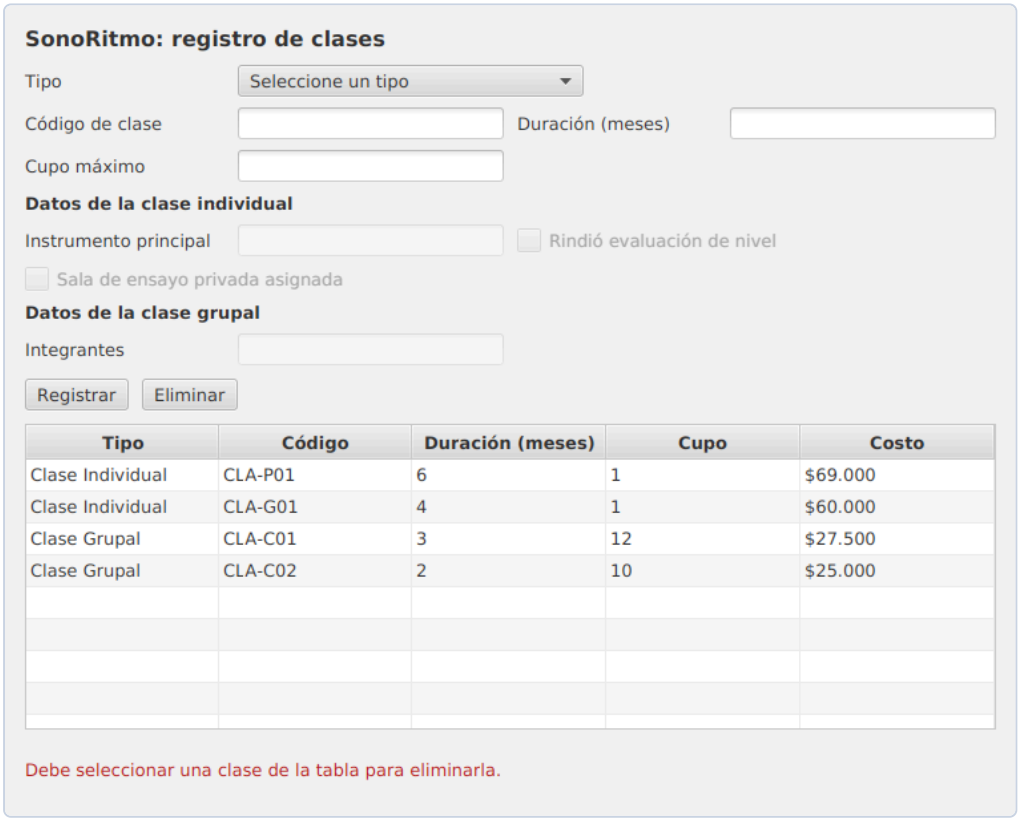


Figura 2. Aviso por falta de selección.

Si un dato del formulario no es válido, el área de mensajes indica el campo con problemas y se mantienen los valores ingresados:

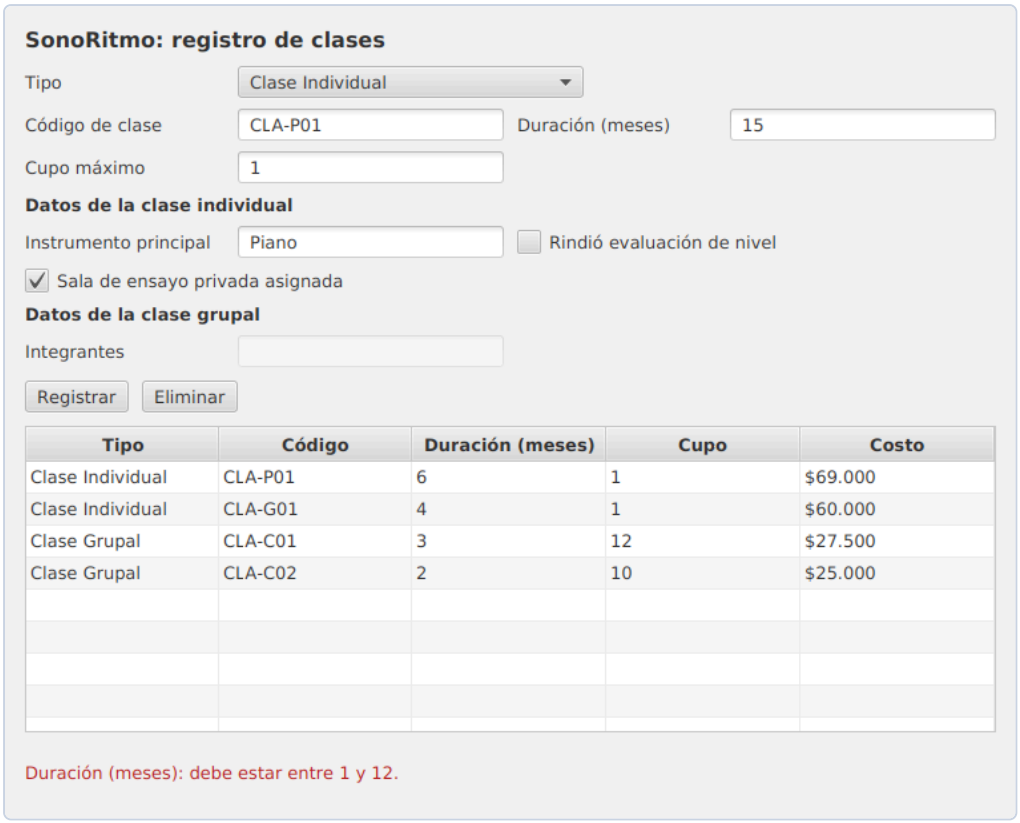


Figura 3. Error de validación en duración (meses).

Verificación de almacenamiento

- Cierra la aplicación y vuelve a abrirla: la tabla debe mostrar los mismos cuatro registros.
- Elimina **CLA-P01**, cierra la aplicación y vuelve a abrirla: CLA-P01 ya no debe aparecer.

Las imágenes son referenciales: la distribución, los textos y el estilo pueden variar, siempre que estén presentes todos los elementos solicitados y que los resultados calculados sean correctos.

7. Paso a paso sugerido (qué hacer, con pistas de API)

Esta ruta dice **qué** construir y **con qué piezas** de JavaFX/Gson, sin mostrar código. Está ordenada para que cada paso se pueda comprobar antes de seguir. Sugerencia de tiempo: **bloque 1** = fases 0 a 4 · **bloque 2** = fases 5 a 8 · **bloque 3** = fase 9 y repaso.

Fase 0 · Preparación inicio

1. Descomprime `SonoRitmo_Base.zip`, ábrelo en IntelliJ IDEA como proyecto Maven (File > Open, *Trust Project*) y verifica que el JDK sea 21.
2. Lee el paquete `model` sin modificarlo: identifica la clase abstracta `ClaseMusical`, sus dos subclases, la interfaz `ConSalaPrivada`, y los métodos que usarás (`calcularCosto()`, `getTipo()`, `asignarSalaPrivada()`) y sus constructores.
3. Ejecuta `Launcher`: debe detenerse con “No se encontró la vista principal”. Es lo esperado; la vista aún no existe.

Pista: Launcher → Application.launch → App.start → FXMLLoader.load.

Fase 1 · Configurar el proyecto IL 2.1

1. Agrega al `pom.xml`, en el lugar que indica el comentario, la dependencia de Gson (groupId `com.google.code.gson`, artifactId `gson`, versión disponible en el laboratorio).
2. Recarga Maven (*Load Maven Changes*) y confirma que el proyecto sigue compilando.
3. Comprueba que entiendes el ciclo de vida: `App` carga el FXML indicado en `VISTA_PRINCIPAL` y lo muestra en un `Stage` con una `Scene`.

Pista: Sección `<dependencies>` del pom; no hace falta tocar `App` ni `Launcher`.

Fase 2 · Diseñar la vista en FXML IL 2.2

1. Crea `src/main/resources/cl/duoc/sonoritmo/view/main-view.fxml` (con Scene Builder o desde IntelliJ).
2. Arma el contenedor: `VBox` raíz, un `GridPane` para el formulario y un `HBox` para los botones.
3. Agrega los controles: un `ComboBox` con los dos tipos, un `TextField` por cada dato de texto o numérico, un `CheckBox` por cada dato sí/no, los botones *Registrar* y *Eliminar*, un `TableView` con una `TableColumn` por columna pedida y un `Label` para los mensajes.
4. Asigna un **fx:id** a cada control y a cada columna, define **fx:controller** con la clase `cl.duoc.sonoritmo.controller.MainController` y enlaza cada botón a su método con **onAction**.

Pista: fx:id, fx:controller, onAction="#nombreMetodo". Usa la política de columnas CONSTRAINED_RESIZE_POLICY si quieres que la tabla llene el ancho.

Fase 3 · Organizar paquetes (MVC) IL 2.4

1. Junto a `model` (entregado) crea los paquetes `controller`, `dao` y `repository` dentro de `cl.duoc.sonoritmo`. La vista es el FXML de `resources`.
2. Define las dependencias permitidas: el controlador usa al repositorio y al modelo; el repositorio usa al DAO y al modelo; **dao, repository y model no importan nada de javafx**.

Pista: Si un import de `javafx.*` aparece fuera del controlador, la separación de capas está rota.

Fase 4 · Persistencia con DAO/Repository IL 2.7

1. Crea la **interfaz** `ClaseMusicalDao` con solo las operaciones que usará el repositorio: `cargarClasesIndividuales()`, `cargarClasesGrupales()`, `guardarClasesIndividuales(lista)`, `guardarClasesGrupales(lista)`. Cada `cargar…` devuelve una lista del subtipo y cada `guardar…` recibe la lista completa.
2. Crea una excepción propia no comprobada (`PersistenciaException`) para los errores de lectura y escritura, con mensaje descriptivo.
3. Crea la clase `ClaseMusicalJsonDao` que **implementa** `ClaseMusicalDao` usando Gson. *Cargar*: si el archivo no existe, devuelve una lista vacía; si existe, lee el JSON como un **arreglo** del subtipo y lo pasa a una lista. *Guardar*: escribe la lista completa en el archivo.
4. Crea `ClaseMusicalRepository`: mantiene en memoria una lista de `ClaseMusical`, recibe en el constructor un `ClaseMusicalJsonDao` (variable de tipo de la clase) y ofrece `cargar()`, `listar()`, `agregar()` y `eliminar()`.
5. Dentro del repositorio, cada `agregar`/`eliminar` guarda de inmediato, separando la lista por subtipo antes de entregarla al DAO. Si guardar falla, deshace el cambio en memoria y relanza el error.
6. Prueba la capa sola (por ejemplo desde un `main` temporal): agrega un clase de cada tipo, revisa los dos JSON y vuelve a cargar.

Pista: Gson.fromJson(lector, Subtipo[].class) + Arrays.asList, Gson.toJson(lista, escritor), GsonBuilder.setPrettyPrinting(), Files.exists, Files.newBufferedReader/Writer, instanceof. Gson necesita los constructores sin parámetros que ya trae el modelo.

Fase 5 · Controlador: inicialización y tabla IL 2.3 · 2.5

1. Crea `MainController` con un atributo **@FXML** por cada `fx:id` (mismos nombres).
2. Instancia el repositorio pasándole un `ClaseMusicalJsonDao`.
3. Crea una `ObservableList<ClaseMusical>` y asígnala a la tabla. Esa lista es solo la presentación; los datos viven en el repositorio.
4. En el método **initialize()** configura cada columna: como el tipo y el costo son valores calculados por el modelo (`getTipo()`, `calcularCosto()`), usa una fábrica de valores de celda que construya el texto de cada fila. Da formato de moneda al valor calculado y de decimales a los números.
5. También en **initialize()**: carga los datos del repositorio capturando el error de persistencia (muéstralo en el área de mensajes) y refresca la tabla.
6. Escucha los cambios del selector de tipo para habilitar solo los campos del tipo elegido (y deshabilitar y vaciar los del otro). Llama a esa misma rutina al iniciar y al limpiar el formulario.

Pista: @FXML, initialize(), FXCollections.observableArrayList(), TableView.setItems, TableColumn.setCellValueFactory + SimpleStringProperty, ComboBox.valueProperty().addListener, Node.setDisable, DecimalFormat.

Fase 6 · Registrar con validación IL 2.3 · 2.6

1. Crea el método enlazado al botón *Registrar* (`onAction`).
2. Antes de crear nada, lee y valida cada campo en el orden de la sección 4: tipo seleccionado, datos comunes y datos del tipo elegido. Convierte los números capturando el error de formato, recorta espacios y compara contra el rango pedido.
3. Cada regla incumplida debe generar un mensaje que nombre **el campo y el motivo**. Si algo falla, no se registra, **no se limpia el formulario** y la aplicación sigue funcionando.
4. Si todo es válido, construye el objeto del subtipo con los constructores del modelo; si el usuario marcó "sala de ensayo privada asignada", invoca `asignarSalaPrivada()`.
5. Entrega el objeto al repositorio, refresca la tabla, informa el éxito (con el identificador y el tipo) y deja el formulario vacío y listo para otro registro.
6. Atrapa tanto los errores del modelo (`IllegalArgumentException`) como los de persistencia; nunca uses `System.out` ni dejes que un error llegue a la consola.

Pista: Integer.parseInt / Double.parseDouble + NumberFormatException, String.trim()/isEmpty(), try/catch, TextField.clear(), ComboBox.setValue(null).

Fase 7 · Eliminar IL 2.3

1. Crea el método del botón *Eliminar*. Obtén la fila seleccionada; si no hay ninguna, avisa en el área de mensajes y termina sin cambios.
2. Si hay una fila, pide al repositorio que la elimine (él actualiza los archivos), refresca la tabla e informa el resultado.

Pista: TableView.getSelectionModel().getSelectedItem() devuelve null cuando no hay selección.

Fase 8 · Mensajes en la interfaz IL 2.3

1. Centraliza los avisos en un solo método que escriba el texto en el `Label`. Cada mensaje reemplaza al anterior.
2. Revisa que registro, dato inválido, eliminación, falta de selección y error de archivos usen el área de mensajes y no la consola.

Pista: Label.setText, wrapText en el FXML para mensajes largos.

Fase 9 · Integración y verificación final IL 2.8

1. Ejecuta con los datos de la sección 6 y compara los valores calculados de la tabla con la columna "esperado".
2. Prueba las tres situaciones de las figuras: eliminar sin selección, dato inválido con valores conservados y registro correcto.
3. Cierra y reabre: la tabla debe mostrar lo mismo. Elimina **CLA-P01**, cierra, reabre y comprueba que no aparece.
4. Borra los dos JSON y abre de nuevo: la aplicación debe iniciar con la tabla vacía y sin mensajes de error.
5. Abre los JSON y verifica que su contenido coincide con la tabla en todo momento.
6. Comprime el proyecto Maven completo (sin `target`) en un ZIP con tu nombre completo.

Pista: Ejecuta con el botón Run de `Launcher` o con `mvn javafx:run`.

8. Autoverificación contra la pauta

Antes de entregar, marca cada punto. La pauta evalúa cada indicador por su propia evidencia en el código, las vistas y la configuración.

Indicador	Peso	Qué debe verse en tu proyecto	OK
IL 2.1	6%	Gson en el pom; la aplicación compila e inicia cargando la vista desde el FXML.	☐
IL 2.2	14%	FXML con todos los componentes del caso; cada componente con `fx:id` + `@FXML` y la vista con su `fx:controller`.	☐
IL 2.3	14%	`initialize()` y manejadores de eventos; mensajes descriptivos dentro de la ventana.	☐
IL 2.4	15%	Paquetes separados; el controlador delega negocio al modelo y datos al repositorio.	☐
IL 2.5	12%	Tabla alimentada por `ObservableList`, con valores calculados y actualización inmediata.	☐
IL 2.6	14%	Validación antes de registrar; campo y motivo informados; valores conservados; formulario limpio tras éxito.	☐
IL 2.7	17%	Interfaz DAO + implementación con Gson + Repository; carga al iniciar y guardado automático tras cada cambio.	☐
IL 2.8	8%	Flujo completo sin errores: inicio con carga, registro, eliminación y persistencia.	☐

Guía de práctica generada como material de estudio. No corresponde a una evaluación oficial.